Compositional Inference: Localized KV-Cache Repair and Specialist Routing

Author: Abhishek Pant
Affiliation: Independent Researcher
Education: MCA (AI/ML specialisation), 2025 graduate
Profile: LinkedIn
Date: October 2026
Repository: github.com/Rusty-user365/AGI
Artifacts: `experiments/path_a_loop.py`, `results/path_a_loop_h5.json`, `findings/2026-10-28-path-a-ensemble-kv-slicing.md`

Abstract

We study a compositional inference framework that combines frozen language-model backbones, learned error localization, bounded interventions, and task-specific low-rank adapters. The intended benefit is modular repair and specialist selection without updating the base weights at inference. This is a proposed alternative to retraining a monolithic system for every specialized behavior; we do not establish a solution to catastrophic forgetting or a replacement for jointly trained mixtures of experts. On five-step arithmetic with Qwen3-0.6B-Base, a frozen rewind-and-token-exclusion loop recovers 73/331 failed chains versus 46/331 for full rewrite, with a 1.70x measured end-to-end speedup. Separately, on the chat-capable Qwen3-0.6B, an exploratory correctness router trained on 200 SecEval questions achieves 57/80 on CyberMetric-80 versus 54/80 for both the base and a cyber adapter. A live implementation reproduces all 80 base answers and routing decisions and all nine fresh-cache specialist outputs; three switches help and none harm on this sample. These are separate experiments with different backbones and tasks, not a demonstrated end-to-end multi-domain DAG system. The public benchmark was previously examined, possible dataset contamination is unknown, and the routing gain is preliminary.

Contributions and results at a glance

Across the arithmetic and exploratory adapter experiments, we report:

1. **Falsification of Heuristic Baselines:** Sequence-level pooled observers fail to select correct trajectories better than the model's own raw token logprob ($\mathrm{AUC} = 0.682$ yields only $+0.004$ selection accuracy over free `max-logprob`, $p = 1.000$). Similarly, natural language hindsight constraints (`# constraint: x is not 7`) are statistically identical to neutral placebos (27.3% vs 27.3%, paired $p = 1.000$).
2. **Step Localization, and the Threshold That Over-Triggers:** A frozen linear probe over SVD-reduced hidden states achieves 0.923 recall on early divergence (Step 1) but collapses to 0.188 on tail steps (Step 4). A cross-step Transformer encoder supplies the complementary tail sensitivity (0.500 Step-4 recall in-sample). An asymmetric Two-Tier rule ($P_{trf}(4) \geq 0.35$) lifts Step-4 recall to 0.812 and Top-1 to 0.718 on the tuning holdout. Out of sample the rule does not collapse — it over-fires. On a frozen blind vault ($N = 331$) it calls Step 4 on 68 rows where only 33 are Step 4, which gives it the **highest Step-4 recall of any arm (**0.667 **vs** 0.545 **for the Transformer) at the lowest Step-4 precision (**0.324 **vs** 0.400**)**, and it pays for that in aggregate Top-1 (0.625 vs 0.677). The smooth softmax ensemble is the robust variant (0.680 Top-1). A scalar cutoff on one class posterior is a recall lever, not a calibrated classifier.

3. **Autonomous Loop Recovery:** Driving a rewind loop with localization and rank-2 first-digit exclusion reaches 29.1% on the $N = 110$ tuning holdout. On the blind vault the frozen loop recovers 22.1% $(73/331)$ against 13.9% $(46/331)$ for full rewrites ($43\text{W}/16\text{L}$, $p = 5.84 \times 10^{-4}$), a $1.59\times$ multiplier. The $2.29\times$ figure sometimes quoted pairs the same loop against a lower-variance draw of the stochastic rewrite baseline and is not the headline.

4. **Systems Efficiency (Two Numbers, Not One):** Physically truncating the attention key-value cache via `DynamicCache.crop(target_len)` cuts per-chain forward tokens from 581.1 to 119.7 (-79.4%) and *generation-slice* latency from $4,498.4$ ms to 173.7 ms ($25.9\times$). The **end-to-end loop** is $2,244$ ms per chain against $4,498.4$ ms $- 2.0\times$ — because ten probe forward passes and the gate readout cost more than the generation they save. $25.9\times$ is a generation-slice measurement and is never a loop claim.

5. **Out-of-Sample Vault Replication & Generalization:** On an independent holdout vault ($N = 331$ unseen broken chains, seed 20261028), frozen CGR achieves 22.1% recovery, significantly beating full rewrites (13.9%, $p = 5.84 \times 10^{-4}$) and decisively outperforming a random-step permutation control (6.9%, $p = 7.64 \times 10^{-11}$). Slicing matched argmax token sequences, with maximum logit difference 4.86×10^{-5}. Loop-level speedup on the vault is $1.70\times$ ($1,934.7$ vs $3,281.9$ ms per chain); the $27.2\times$ in the systems block is the generation slice alone.

6. **Live specialist routing:** On a previously inspected 80-question public cybersecurity benchmark, the frozen SecEval-trained binary router scores 57/80 versus 54/80 for the base and the cyber adapter; a per-question oracle scores 61/80. The live loop regenerates all base answers, reproduces all nine selected cyber outputs after full-prompt prefill from an empty cache, and observes three useful and zero harmful switches in this sample. Neither the base nor adapter weights are updated during inference, although the router heads were supervised-trained. This validates a narrow implementation, not universal safety or general multi-step compositional recovery.

Introduction

A long generated solution may contain many steps that are useful even after a later step fails. Consider an agent constructing a program in 20 steps: a defect discovered at step 20 might originate in step 6, while earlier validated work and some independent artifacts remain useful. A full rewrite discards this work; repeated attempts by the same model may also reproduce the same error. Other systems can checkpoint or edit their work, so full regeneration is a comparison baseline, not an unavoidable property of all agents.

This paper investigates whether an inference controller can keep the verified prefix, locate a suspected failure, and repair only the necessary portion. A dependency graph records which artifacts depend on the changed step: descendants require revalidation or regeneration, while independent artifacts can be preserved. The attention cache has a stricter constraint: even preserved artifacts that occur after a changed prefix may need their tokens replayed, because their cached states were computed in the old context. The controller therefore records the pre-error token boundary, truncates the cache to that boundary, and rebuilds later state rather than assuming artifact independence implies cache independence.

The measured arithmetic experiments show that physical KV-cache truncation is possible for the tested model and cache implementation. On a 110-chain tuning holdout on a GTX 1050 4 GB GPU, the cached **generation slice** was 25.9 times faster than a full rewrite; including the observer and gate, the measured full loop was 2.0 times faster. On the 331-chain vault the full-loop speedup was 1.70 times. These figures are specific to the reported tasks, model, hardware and accounting; they are not measurements of code-agent repair or frontier models. Similarly, the tested textual hindsight constraint performed no better than its neutral placebo on the arithmetic task; this does not establish that all language-based self-correction is ineffective.

The architecture also explores a second route to a different repair: a separately trained specialist adapter can generate a candidate after a verifier rejects a result. Accepted specialist tokens can be replayed under the base model through a bridge before continuation. We report a natural base-failure/coder-pass example and a separate four-node coder/cyber card fixture, including its failed first coder attempt and verifier-feedback retry. We also report a live binary cyber-router experiment. These results test components and small integrations; they do not yet establish a complete 20-step delayed-failure graph, a universally reliable error detector, or safe deployment across domains.

The motivation is broader than any one model size: if failures can be identified and useful state preserved, localized repair may avoid unnecessary recomputation. Whether the same approach helps larger models, frontier agents or different hardware is an open empirical question. We do not assume that a larger model always repairs an error, or that adapters and caches can be interchanged between different base models.

---

1. Preliminaries & Rigorous Negative Controls

Autonomous error recovery in language models requires answering three nested questions:

- *Detection:* Does the model's internal representation signal when a mistake occurred?
- *Localization:* Can an observer determine *which* discrete step was the first point of divergence?
- *Intervention:* Can the generation be steered away from the error without re-running sound prefix computation?

Before building the step-localizer, we tested and falsified three common hypotheses in the literature.

1.1 Sequence-Level Observer Ranking Null

A persistent conjecture is that an auxiliary sequence-level observer head trained on pooled sequence representations can rerank sampled model outputs more effectively than token logprobs.

We evaluated a 2-layer sequence-level MLP head across $n = 500$ multi-step arithmetic tasks ($1,096$ wrong-but-well-formed holdout candidates). While the head achieved above-chance discrimination ($\mathrm{AUC} = 0.682\ [0.573, 0.782]$ vs permutation control 0.496), it failed completely as a generation selector:

$$\text{Head Selection Top-1: } 0.560 \quad \text{vs} \quad \text{Max-Logprob: } 0.556 \quad (+0.004,\ 12\text{W}/11\text{L},\ p = 1.000)$$

The observer added zero marginal value over the model's own intrinsic token probabilities, which are free. Sequence pooling collapses the localized signal: an early catastrophic arithmetic failure produces similar average activation drift to a subtle late rounding error.

1.2 Sub-1B Symbolic State-Tracking Floor

We probed whether small autoregressive models (0.6B–1.5B) maintain internal symbolic state tracking on multi-entity physical state tasks (e.g., tracking colored balls across boxes). On single 1-ball displacement tasks:

- `Qwen3-0.6B-Base`: 22.5% accuracy (2 boxes), collapsing to 2.5% on 3–4 boxes.
- `Qwen2.5-1.5B-Instruct`: 2.5% raw greedy, 15.0% in chat format.

When Step 1 fails in 77.5% of instances, the ground-truth first-error index is degenerately $t = 0$, rendering step localization ill-defined. To establish a valid instrument, we designed `arith_steps`: sequential assignment chains with strict syntax compliance (`x = 5`, `y = x + 3`, ...), where Step 0 is virtually error-free (0.991) and error divergence spreads across later steps.

1.3 Capacity Control & In-Distribution Steering Nulls

In early modular plastic memory experiments (CGR/FiLM architecture), the apparent gating gain turned out to be capacity. Against a LoRA-only checkpoint it looked decisive — $+0.135$ $[0.060, 0.210]$, significant — but that control trained $442{,}368$ parameters against CGR's $8{,}149{,}661$, so the comparison confounded conditioning with a $19\times$ capacity gap (`findings/2026-10-25-steering-capacity-control.md`). Raising the baseline to a matched $22{,}324{,}637$ trainable parameters, with the same corpus, seed and schedule, moved it from 0.245 to 0.350 while CGR moved from 0.380 to 0.385. The matched paired difference is $+0.035$ $[-0.045, +0.115]$ — **not significant** — and the one significant difference runs the wrong way: the steered arm damages edit quality by -0.156 $[-0.240, -0.065]$ while restoring the thesis no more reliably than the control.

Erratum (2026-10-29): an earlier draft reported this control as "$+0.105$ vanished when matched against parameter-equivalent LoRA adapters (8.15M vs 442K)". No artifact contains that pairing: `8.15M` vs `442K` is the *unmatched* comparison, which produced $+0.135$, not $+0.105$, and $+0.105$ appears nowhere in `results/`. The matched control is 22.3M vs 22.3M. See Errata at the end.

Conclusion: Heuristic rerankers, ungrounded prompt tweaks, and parameter-unmatched gates yield illusory gains. Progress requires explicit step-level credit assignment grounded in physical key-value states.

---

2. Un-Pooled Step Localization & The Two-Tier Architecture

To localize errors, we construct an un-pooled representation $\mathbf{H} \in \mathbb{R}^{H \times d}$, where row t contains the mean hidden state across the tokens of the emitted value at step t ($H = 5, d = 1024$).

2.1 Why Confidence and Heuristics Fail

We evaluated four non-learned baselines on $N = 110$ held-out broken chains:

Baseline Arm	Mechanism	Top-1 Accuracy	Balanced Accuracy	Description
`chance`	Uniform $1/H$	0.200	0.200	Theoretical random guess
`modal_prior`	Always predict Step 1	0.355	0.200	Empirical majority step
`argmin_logprob`	$\arg\min_t \log P(v_t)$	0.400	0.293	Predicts Step 4 in 93/110 cases
`poslogprob`	Position bias + slope	0.445	0.347	10 params; non-modal accuracy collapses to 0.141

Per-step token confidence ($\log P$) is dominated by sequence position rather than correctness: token entropies naturally drift as sequence length increases. Consequently, `argmin_logprob` simply predicts the final step ($t = 4$) regardless of where the arithmetic actually derailed.

2.2 Linear vs. Transformer Localizers: Complementary Defect Profiles

We trained two observer families on frozen SVD_{32} projections of the hidden states:

1. **Linear Probe:** Independent linear classification per step with no cross-step attention.
2. **Transformer Encoder:** A 1-layer bidirectional self-attention encoder over the H step embeddings.

While both models achieve identical aggregate Top-1 accuracy (0.618), their per-step error profiles are non-overlapping:

```
Step 1 Divergence: [Linear Recall: 0.923]  >>>  [Transformer Recall: 0.769]  (Linear +0.154)
Step 4 Divergence: [Linear Recall: 0.188]  <<<  [Transformer Recall: 0.500]  (Trf +0.312)
```

The linear localizer defaults to Step 1 in 59% of instances, failing catastrophically on late-chain errors (Step-4 recall collapses to 0.188). Conversely, the Transformer's bidirectional attention mechanism resolves cross-step relational dependencies, allowing it to accurately identify when Step 4 deviates from an otherwise coherent prefix.

2.3 The Threshold Over-Fires Out-of-Sample

Because the error distributions are complementary, a threshold on the transformer's step-4 posterior can lift tail recall without training a large unified model. It does so on the split the threshold is chosen on, and not out of sample — which is the point of this section:

$$\hat{t} = \begin{cases} 4, & \text{if } P_{trf}(t=4) \geq 0.35 \\ \arg\max_t P_{lin}(t), & \text{otherwise} \end{cases}$$

Table 1: Localizer Diagnostic Performance on the Tuning Holdout ($N = 110$, 5 Seeds) — In-Sample for τ

Architecture	Top-1 Accuracy	Balanced Accuracy	Step 1 Recall ($n = 39$)	Step 2 Recall ($n = 23$)	Step 3 Recall ($n = 13$)	Step 4 Recall ($n = 32$)
Linear Majority (SVD_{32})	0.618	0.515	0.923	0.696	0.769	0.188
Transformer Majority	0.618	0.505	0.769	0.565	0.692	0.500
Ensemble (Softmax Mean)	0.691	0.571	0.897	0.783	0.769	0.406
Two-Tier Localizer	0.718	0.559	0.846	0.522	0.615	0.812

The Two-Tier localizer achieves a $4.3\times$ **gain in Step-4 recall** ($0.188 \rightarrow 0.812$), lifting aggregate Top-1 localization accuracy from 0.618 to 0.718 ($+10.0\%$ absolute) — **on the split used to choose $\tau = 0.35$.**

That last qualification is load-bearing. On the frozen blind vault ($N = 331$, Table 5) the same rule scores 0.625 Top-1, below the Transformer alone (0.677) and below the softmax ensemble (0.680). It is not failing to localize tail steps, though: it calls Step 4 on 68 of 331 rows when only 33 are Step 4, which lifts Step-4 recall to 0.667 (Transformer 0.545, ensemble 0.485) and drops Step-4 precision to 0.324 (0.400, 0.471). A single scalar threshold on one class posterior is a fitted recall lever, and out of sample it over-fires. The ensemble has no threshold and generalizes; the Two-Tier rule is reported here as the tuning-set maximum, not as the recommended model.

2.4 Dissecting the Inductive Bias: Relational Attention vs. Parameter Capacity Confound

A skeptical reviewer might reasonably ask the exact question that falsified the Phase 1 auxiliary-head claim: *Is the Transformer localizer winning because of cross-step attention, or merely because it possesses more trainable parameters than a linear projection?*

The empirical evidence rules out the parameter-capacity hypothesis by examining **where** the performance divergence occurs:

1. **Parameter Count Does Not Explain Per-Step Asymmetry:**
   If parameter volume alone explained the Transformer's edge (0.677 vs 0.604 Top-1 on blind vault data), a wider multi-layer perceptron operating on steps independently should have matched it. Instead, the models exhibit complementary functional specializations:
   - **The Linear Probe's Early-Step Competence:** The linear probe achieves 0.923 **recall on Step 1**. When an early error occurs, it introduces severe, immediate mathematical divergence into the local hidden state h_1. A single linear hyperplane easily partitions this perturbed activation from normal prefixes; additional parameters provide no marginal utility.
   - **The Transformer's Tail-Step Competence:** The Transformer's advantage is concentrated exclusively on late-chain divergence (Step-4 recall jumps from 0.188 to 0.500 on holdout, and from 0.394 to 0.545 on the vault re-fit).
2. **The Residual Stream Drift Problem:**
   In deep autoregressive sequences, later-step activations accumulate residual stream variance, positional encodings, and multi-token attention mixing. By Step 4, even an arithmetic step that evaluated correctly possesses a complex, drifted activation profile that mathematically overlaps with an erroneous step when projected in isolation. Consequently, an isolated linear probe loses discriminative sharpness and collapses to the empirical majority prior (Step 1).
3. **Cross-Step Attention as Relational Inductive Bias:**
   The 1-layer Transformer encoder computes all-to-all self-attention across the H step embeddings:
   $$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$
   This allows Step 4's representation to attend backward over Steps 1, 2, and 3. By performing relational comparison—verifying that the representations of Steps 1–3 are internally coherent and match task priors—the attention head isolates the anomaly specifically to Step 4. Relational context, not raw weight volume, resolves tail ambiguity.
4. **The Principled Two-Tier Synthesis:**
   The Two-Tier architecture directly operationalizes this functional division: Tier 1 (Linear) executes fast, parameter-minimal cuts on early steps, while Tier 2 (Transformer) acts as a targeted override only when tail-step posterior probability exceeds threshold ($P_{trf}(4) \geq 0.35$).

---

3. Mechanism of Repair: Hindsight Placebo vs. Token Exclusion

Localization specifies *where* to intervene, but not *how*. We evaluated seven intervention mechanisms under an **Oracle Localizer** (intervening at the true divergence step t^*) across all 110 broken holdout chains.

Table 2: Oracle Intervention Taxonomy ($N = 110$ Broken Chains, Qwen3-0.6B-Base)

Arm	Perturbation Mechanism	Recovered / N	Rate	95% Wilson CI	Re-emitted Bad Value	Paired vs Rewrite
`full_rewrite`	Regenerate all 5 steps ($T = 0.8$)	20 / 110	0.182	$[0.121, 0.264]$	—	—
`rewind_greedy`	Greedy replay from t^*	17 / 110	0.155	$[0.099, 0.234]$	0.618	$6\text{W}/9\text{L}, p = 0.61$

Arm	Perturbation Mechanism	Recovered / N	Rate	95% Wilson CI	Re-emitted Bad Value	Paired vs Rewrite
`resample`	Sample step t^* ($T = 0.8$), greedy after	22 / 110	0.200	$[0.136, 0.284]$	0.445	$15\text{W}/13\text{L}, p = 0.85$
`hindsight`	Prompt: `# constraint: x is not <bad>`	30 / 110	0.273	$[0.198, 0.363]$	0.136	$23\text{W}/13\text{L}, p = 0.13$
`hindsight_neutral`	Prompt: `# constraint: none` (Placebo)	30 / 110	0.273	$[0.198, 0.363]$	0.500	$20\text{W}/10\text{L}, p = 0.10$
`rank2`	**Forbid bad first digit via logit mask**	**35 / 110**	0.318	$[0.239, 0.410]$	0.000	$22\text{W}/7\text{L}, p = 0.008$
`oracle_force`	Inject ground-truth value at t^*	94 / 110	0.855	$[0.777, 0.908]$	—	Ceiling (79.5% non-trivial)

3.1 The Greedy No-Op and Prompting Placebo

1. **Greedy Replay Fails:** `rewind_greedy` reproduces the identical erroneous digit in 61.8% of instances, achieving a lower recovery rate than a fresh rewrite (0.155 vs 0.182). Slicing back to the error point without perturbation is mathematically inert.
2. **The Hindsight Prompt is an Inert Perturbation:** Natural language hindsight constraints (`# constraint: e is not 7`) recover exactly $30/110$ chains (0.273). However, the neutral placebo line (`# constraint: none`) **also recovers exactly $30/110$ chains (0.273)**. In a direct paired comparison, the real constraint and the placebo tie:

$$30/110 \quad \text{vs} \quad 30/110 \quad (15\text{W}/15\text{L},\ p = 1.000)$$

The prompt text suppresses the named token (re-emission drops to 13.6%), but fails to steer generation toward truth; the model simply selects an alternative erroneous digit. The apparent benefit of "hindsight prompting" is entirely attributable to the syntactic whitespace and token shift of adding an extra prompt comment.

3.2 Rank-2 Logit Exclusion Beats Sampling

Physical logit masking (`rank2`) sets the logits of the first digit of the rejected value to $-\infty$:

$$\tilde{z}_i = \begin{cases} -\infty, & \text{if } i \in \text{Digits}(\text{bad_val}[0]) \\ z_i, & \text{otherwise} \end{cases}$$

This forces the model to take its second-highest probability branch without relying on instruction following. `rank2` recovers 31.8% of broken chains ($22\text{W}/7\text{L}, p = 0.008$), completely eliminating bad-value re-emission (0.0%).

---

4. End-to-End Autonomous Loop & Systems Benchmark

We wired the Two-Tier localizer, the `rank2` intervention, and an "Accept-if-Cleaner" gate into a fully automated rewind loop operating on un-annotated, raw model completions.

4.1 The Accept-if-Cleaner Gate

When an observer mis-localizes a late defect to Step 1, applying `rank2` risks corrupting an already-correct prefix. We implemented a confidence guard:

- Let lp_{orig} be the logprob of the original emitted token at $\hat{t}$, and lp_{edit} be the logprob under rank-2 masking.
- If the original step was confident ($lp_{orig} > -0.25$) and the edit sharply degrades confidence ($lp_{edit} < lp_{orig} - 0.35$), the edit is **rejected**, and the prefix is preserved.

In empirical holdout testing, the gate rejected $23/110$ candidate edits. The gated and ungated Two-Tier arms are **numerically identical** ($32/110$, $34{,}046$ tokens, $246{,}826$ ms), because the rejected rows contributed zero recoveries when ungated. The same holds on the vault: $81/331$ rejected, 0 recoveries among them, and the gated arm is the only arm run there. So the gate is a *conservative filter that cost nothing*, not a demonstrated recovery gain: it removes edits that would not have helped, and it buys precision ($32/87 = 36.8\%$ recovery among accepted rows on $N = 110$) plus skipped compute. An auxiliary probe evaluated falling back to the localizer's second-highest ranked step ($\hat{t}_2$) on those 23 rejected rows: fallback hit the true step in $3/23$ instances but converted $0/23$ chains into additional recoveries, which is why terminating on rejection is the compute policy we use.

Table 3: End-to-End Autonomous Loop Recovery on the Tuning Holdout ($N = 110$)

Autonomous Arm	Localization Model	Intervention	Recovered / N	Success Rate	95% Wilson CI	$\hat{t} = t^*$ Exact	Accepted / N
`full_rewrite`	None (Sample $T = 0.8$)	Full sequence	14 / 110	0.127	$[0.077, 0.202]$	—	—
`linear_ungated`	Linear Majority	`rank2`	24 / 110	0.218	$[0.151, 0.304]$	0.618	—
`ensemble_ungated`	Ensemble Average	`rank2`	31 / 110	0.282	$[0.206, 0.372]$	0.691	—
`two_tier_ungated`	**Two-Tier Rule**	`rank2`	**32 / 110**	0.291	$[0.214, 0.382]$	0.718	—
`ensemble_gated`	Ensemble Average	`rank2` + Gate	31 / 110	0.282	$[0.206, 0.372]$	0.691	66 / 110
`two_tier_gated`	**Two-Tier Rule**	`rank2` + **Gate**	**32 / 110**	0.291	$[0.214, 0.382]$	0.718	**87 / 110**

Autonomous loop recovery jumps from 12.7% (rewrite) and 21.8% (linear) to 29.1% ($32/110$). Because the theoretical ceiling of `rank2` under an omniscient oracle is 31.8% ($35/110$, Table 2), the Two-Tier autonomous loop captures:

$$\frac{29.1\%}{31.8\%} = \textbf{91.5\%} \text{ of the theoretical oracle repair ceiling}$$

This ratio is a property of the $N = 110$ tuning draw and should be read as such: the oracle ceiling was not re-measured on the vault, so no vault ceiling ratio is claimed. The vault result (Section 4.3) stands on its own paired comparison against full rewrites.

4.2 Physical KV-Cache Slicing (`DynamicCache.crop`)

Prior rewind experiments re-ran forward passes over the entire prompt and prefix. To test the systems claim, we integrated physical key-value cache slicing using HuggingFace's `DynamicCache.crop(target_len)`.

During autoregressive generation, past key-value projections for prompt tokens and verified prefix steps are retained in memory. When the localizer triggers a rewind to step $\hat{t}$:

1. The token length $L_{\hat{t}}$ corresponding to the prompt up to step $\hat{t} - 1$ is computed.
2. The past key-value tensors are truncated in-place: `cache.crop(L_target)`.
3. Generation resumes from step $\hat{t}$ without re-evaluating preceding tokens.

Table 4: Systems Benchmark, Generation Slice Only ($N = 110$ Chains, GTX 1050 4GB)

Generation Arm	Mean Tokens / Chain	Mean Latency / Chain	FLOPs / Token Reduction	Latency Reduction	Relative Speedup
Full Rewrite ($T = 0.8$)	581.1	$4,498.4$ ms	Baseline	Baseline	$1.0\times$
Un-cached Fresh Rewind	379.5	$2,732.2$ ms	-34.7%	-39.3%	$1.6\times$
KV-Cache Truncated Rewind	119.7	173.7 ms	-79.4%	-96.1%	$25.9\times$

Compared to standard full-sequence regeneration, surgical KV-cache truncation eliminates 79.4% of forward tokens and reduces *generation-slice* wall-clock latency by 96.1% (a $25.9\times$ speedup). Even against naive un-cached rewinds, retaining the prefix key-value states delivers a $15.7\times$ speedup ($2,732.2$ ms $\rightarrow$ 173.7 ms).

This table is not loop latency. Measured end to end, the same arm in Table 3 cost $246,826$ ms for 110 chains $- 2,244$ ms **per chain against** $4,498.4$ ms **for a full rewrite, i.e.** $2.0\times$. The difference is the observer: ten probe forward passes plus the post-edit readout needed by the Accept-if-Cleaner gate. On the vault the same accounting gives $1,934.7$ ms vs $3,281.9$ ms per chain, $1.70\times$. The honest systems claim is therefore two-tiered: *generation* is 25.9-$27.2\times$ cheaper, the *loop* is 1.7-$2.0\times$ cheaper. Quoting the first as the second would overstate the system benefit by an order of magnitude.

4.3 Out-of-Sample Vault Replication ($N = 331$) and Anti-Self-Deception Controls

To eliminate post-hoc threshold overfitting concerns on small holdout splits, we conducted a blind vault replication on an independent test split (`seed=20261028`, 140 tasks, 560 candidates, yielding 331 **complete broken chains**) with all probe weights, SVD projection matrices, and decision thresholds permanently frozen ($\tau_{trf} = 0.35, lp_{orig} = -0.25, \Delta lp = -0.35$).

Table 5: Blind Vault Replication and Permutation Control ($N = 331$ Unseen Chains)

Generation Arm	Intervention Mechanism	Recovered / N	Rate	95% Wilson CI	$\hat{t} = t^*$ Exact	Accepted / N
`full_rewrite` ($T = 0.8$)	Regenerate entire chain	46 / 331	0.139	$[0.106, 0.180]$	—	—
`random_t_rank2`	Uniform random $\hat{t} \in \{1 \dots 4\}$ + `rank2`	23 / 331	0.069	$[0.047, 0.102]$	0.245	—
`two_tier_gated` **(CGR)**	**Two-Tier Probe + Gate +** `rank2`	**73 / 331**	0.221	$[0.179, 0.268]$	0.625	**250 / 331**

1. **Localization Generalizes; the Hard Threshold Over-Triggers (Rule 1, reframed):** The relational inductive bias holds out of sample: Transformer Majority Top-1 reached 0.677 (vs 0.604 for Linear), up from 0.618 on the tuning holdout, and the Ensemble Average held at 0.680 — a 0.011 drop, not a collapse. The **Two-Tier rule** lands at 0.625 Top-1, below both, and the reason is

not that it stopped finding tail errors. It is that $\tau_{trf} = 0.35$ is a recall lever: it calls Step 4 on 68 **of** 331 **rows where only** 33 **are Step 4**, buying the highest Step-4 recall of any arm at the lowest Step-4 precision.

Arm	Top-1 (published fit)	Step-4 Recall (re-fit)	Step-4 Precision (re-fit)	Rows called Step 4 (re-fit)
Linear Majority	0.604	0.394	0.342	38
Transformer Majority	0.677	0.545	0.400	45
Ensemble (softmax mean)	0.680	0.485	0.471	34
Two-Tier ($\tau = 0.35$)	0.625	0.667	0.324	68

$\tau_{trf} = 0.35$ was selected on the same 110 rows where it scored 0.812 Step-4 recall and 0.718 Top-1; on unseen seeds it is calibration drift, not a law. The transferable finding is the *complementarity* of linear and cross-step attention, and the fact that a hard cutoff converts it into a precision/recall trade rather than a free gain. The ensemble keeps the complementarity without the cutoff.

Metric erratum (2026-10-29): the previous version of this item read "0.625 / 0.410 Step-4 recall, below both". The 0.410 was a per-step **precision**, produced by calling `per_step_recall(lab, pred)` where the signature is `per_step_recall(pred, lab)`. On recall the Two-Tier arm was never below the others; it was above them. Top-1 above is from the published fit because that fit drove the loop in Table 5; the recall, precision and call-count columns are a deterministic CPU re-fit (two runs identical), because the 2026-10-28 fit ran on a different torch build and stored no predictions. The re-fit reproduces Linear and Transformer Top-1 within 0.01 and moves the two threshold-dependent arms by 0.027 (Two-Tier $0.625 \to 0.598$), which is the same `P_trf(4) >= 0.35` fragility argued for here. Both fits agree on the ordering that matters: Two-Tier maximizes Step-4 recall and minimizes Step-4 precision. See Errata.

2. **Defeating the Permutation Control (Rule 3 PASS):**
   Slicing back to a uniformly random step with rank-2 exclusion (`random_t_rank2`) collapses to 6.9% **recovery** $(23/331)$—performing at roughly half the recovery rate of doing a standard full rewrite (13.9%). Arbitrary rewinds actively destroy sound prefixes. In contrast, Two-Tier CGR achieves 22.1% $(73/331)$. The paired McNemar test demonstrates decisive statistical superiority:

   $$\mathbf{57}\ \text{Wins}\ /\ \mathbf{7}\ \text{Losses} \quad (p = 7.64 \times 10^{-11})$$

   The paired odds ratio of $8.14:1$ shows a recovery advantage over this particular random-step control; it does not isolate all causal mechanisms.

3. **Replicating Out-of-Sample Systems Savings (Rule 2, qualified):**
   CGR significantly outperforms full-chain rewrites (22.1% vs 13.9%, $43\text{W}/16\text{L}$, $p = 5.84 \times 10^{-4}$) while slashing forward tokens from 580.6 to 120.1 (-79.3%) and per-chain latency from $3,281.9$ ms to 120.7 ms—a $27.2\times$ **generation-slice speedup**, with the end-to-end loop at $1,934.7$ ms vs $3,281.9$ ms per chain ($1.70\times$).

4. **Argmax-Identical Slicing Verification (Rule 4 PASS):**
   To check cache slicing for numerical drift or token divergence, we benchmarked `DynamicCache.crop` against un-cached recomputations across 20 validation sequences. The observed maximum absolute logit difference was $\max|\Delta\text{logit}| \leq 4.86 \times 10^{-5}$, below the 10^{-4} comparison tolerance, with 100% identical argmax token sequences. This is not a claim that the logit difference is below fp32 machine epsilon.

4.4 Long-Horizon Scaling ($H = 10$) & The Generative Horizon Boundary (H^*)

To establish the limits of test-time KV rewinds as reasoning chains deepen, we scaled the arithmetic task from $H = 5$ to $H = 10$ assignments on `Qwen3-0.6B-Base` ($N = 75$ broken holdout chains, `results/verify_h10_scaling.json`).

Table 6: Horizon Scaling Diagnostics and Recovery at $H = 10$ ($N = 75$)

Evaluation Arm	Mechanism	Top-1 Localization	Full-Chain Recovery	95% Wilson CI	Mean Forward Tokens
`linear_localizer`	SVD-32 Linear Probe (5 seeds)	0.520 (Chance 0.100)	—	—	—
`full_rewrite`	Regenerate from Step 0 ($T = 0.8$)	—	0 / 75 (0.000)	$[0.000, 0.049]$	$1,656.2$
`in_context_oops`	Verbal retry prompt without KV crop	—	0 / 75 (0.000)	$[0.000, 0.049]$	220.6
`two_tier_gated_1rewind`	Two-Tier CGR + `rank2` + Gate	0.520	0 / 75 (0.000)	$[0.000, 0.049]$	165.0
`two_tier_gated_max2`	Sequential max-2 rewinds	—	0 / 75 (0.000)	$[0.000, 0.049]$	171.9

1. **Robust Localization vs. Multiplicative Rollout Decay:** While the localizer reliably identifies the erroneous step with 52.0% **accuracy** ($5.2\times$ over chance), every generation arm collapsed to 0.0% full-chain recovery. Because per-step conditional accuracy on sub-1B models hovers at $p \approx 0.35$–0.50, the joint probability of completing the remaining $10 - t$ steps correct without error decays exponentially: $\prod_{k=t+1}^{10} p_k \leq (0.50)^6 \approx 0.015$.
2. **Identification of the Autoregressive Horizon Boundary (H^*):** For small language models, unassisted autoregressive arithmetic possesses an empirical horizon ceiling at $H^* \approx 6$–8. Beyond H^*, error recovery cannot rely solely on test-time branch exploration; it strictly necessitates external symbolic tool invocations (e.g. calculator tool) or checkpoint scaling to preserve late-step fidelity.
3. **Token Conservation Does Not Become Latency:** Even at the generative capacity ceiling, surgical KV-cache truncation reduces token forward evaluation by 90.0% (165.0 vs $1,656.2$ tokens). Wall-clock latency, however, moves from $7,502.7$ ms to $6,399.2$ ms — $1.17\times$ — because un-batched single-sequence inference at this scale is dominated by framework launch and prefill overhead rather than decode FLOPs. A 90% token reduction therefore does not imply a 90% latency reduction, and at $H = 10$ the loop is only marginally faster while recovering nothing.

---

5. Limitations & Remaining Headroom

While Path A substantially resolves the localization bottleneck and establishes that KV slicing is real, four claims required correction after the audit and are reported as such above: the Two-Tier threshold is a recall lever rather than a joint classifier, the vault Step-4 column was a precision mislabelled as recall, the loop speedup is 1.7-2.0x rather than 27x, and the rewrite multiplier is 1.59x rather than 2.29x. Beyond those corrections, several challenges remain.

1. **The Rank-2 to Oracle Gap (31.8% vs 79.5%):**
   Under an oracle that directly injects the ground-truth value, 79.5% of non-trivial broken chains are restored. Under `rank2`, recovery caps at 31.8%. First-digit exclusion guarantees that the model will

not repeat its previous answer, but when numbers span multi-digit ranges ($[1, 99]$), excluding one digit still leaves multiple plausible incorrect candidates. Closing this remaining headroom requires token-level value steering or beam-guided branch exploration at step $\hat{t}$.

2. *The Autoregressive Horizon Ceiling ($H^ \approx 6-8$):*
   While the Two-Tier hidden-state localizer continues to isolate defects accurately at $H = 10$ (52.0% vs 10.0% chance), full-chain recovery collapses to 0.0% across all arms due to multiplicative rollout compounding ($(0.45)^6 < 0.01$). Slicing past the error point succeeds locally, but completing long downstream sequences requires coupling CGR with external symbolic tool execution or larger parameter checkpoints to prevent subsequent divergence.
3. **Hybrid Architecture Compatibility:**
   In linear-attention or state-space models (e.g., Mamba, LFM3), key-value caching is replaced by recurrent convolutional and state matrices. Rewinding in such architectures requires rolling back the hidden recurrent state h_t rather than slicing a sequence cache.

5.1 Scaling Hypothesis: Longer Tasks and Different Models

The observed $H = 10$ recovery failure motivates studying whether stronger generators, external tools or different repair policies can extend the usable horizon. Model capacity is one plausible contributor, but the current experiments do not identify it as the sole cause; localization errors, intervention quality, task difficulty and accumulated downstream errors may also matter. Neither successful large-model repair nor a universal horizon threshold follows from the present measurements.

A future evaluation could compare full rewrite, checkpointed continuation, localized base repair and specialist-assisted repair on the same long-context tasks across model sizes, while measuring verifier quality, retained-prefix correctness, full-loop latency, memory use and task success. Context-retrieval or experience-memory interventions could be tested separately with controls, so an improvement is not attributed to cache truncation alone. The existing arithmetic results and isolated adapter bridges are useful tests of particular mechanisms, not evidence that the complete approach will transfer unchanged to frontier systems.

Cache operations and adapter switching depend on model architecture, tokenizer, cache representation and runtime. A LoRA adapter trained for one backbone cannot simply be attached to a different-size backbone, and caches from different models or policies must not be assumed compatible. Full specialist prefill or a separately validated bridge is required where applicable. Any claimed memory capacity, loading latency, end-to-end speedup or benefit at larger scale must be measured on the target setup.

---

6. Dynamic Adapter Routing and the Bridge Protocol

> **Status: preliminary coder integration and single-question cyber routing; multi-domain DAG efficacy pending.** The CPU prototype measures coder switching and base Bridge coherence on disclosed tasks (§6.3), and a learned cyber correctness router in a separate single-question experiment (§6.5). It does not establish multi-domain DAG recovery, deployment speedup, or a 4 GB memory bound.

Full-checkpoint escalation was measured and closed. On the $N = 110$ ladder no deployable actuator beat `rank2` (`findings/2026-10-28-level2-actuator-ladder.md`), and the memory arithmetic is unforgiving on the target hardware: the AR backbone occupies ∼1.2 GB and a 1.5–2.5B-class specialist a further ∼2.2 GB, so a two-checkpoint policy spends ∼3.4 GB before activations on a 4 GB budget. The alternative is to swap low-rank adapter weights (∼10–50 MB) instead of a parameter backbone. The CGR line already produces adapters in that band — the matched LoRA control trains 22.3M parameters (`findings/2026-10-25-`

steering-capacity-control.md), ≈ 45 MB at fp16. A separate coder LoRA checkpoint is now available on the chat-capable Qwen/Qwen3-0.6B, rather than the arithmetic experiments' Qwen3-0.6B-Base. Its preliminary integration is reported in §6.3; specialist quality is a separate question from its size or successful loading.

6.1 KV-Cache Coherence Under Adapter Intervention

A fundamental systems constraint in recurrent adapter intervention is **Key-Value Cache Coherence**: updating attention projections ($\Delta W_q, \Delta W_k, \Delta W_v$) or feed-forward layers (ΔW_{MLP}) alters the activation distribution, rendering cached tensors mathematically misaligned with subsequent decoding steps. Two operational paths can manage this boundary.

Path 1 — Query-Restricted Adaptation (in-stream persistence). Adapters are constrained during training to low-rank perturbations of query and MLP projections ($\Delta W_q, \Delta W_{\text{MLP}}$), keeping the key-value projection weights (W_k, W_v) strictly frozen. When an error is localized at index $\hat{t}_{\text{min}}$, the controller crops the cache via DynamicCache.crop to $\hat{t}_{\text{min}} - 1$. Because W_k and W_v are invariant, this design leaves their projection matrices unchanged. It does **not** imply cache equivalence: Q/MLP changes can alter the hidden states from which subsequent layers compute keys and values. Attending to a base-produced prefix under specialist weights is therefore a mixed-policy computation that must be compared with full-prefix specialist prefill. Keeping the adapter active to termination avoids another switch but does not itself prove coherence or equivalent generation quality.

The crop operation itself is measured, not assumed: on 20 validation sequences DynamicCache.crop reproduced un-cached recomputation to $\max |\Delta\text{logit}| \leq 4.86 \times 10^{-5}$ with 100% bit-identical argmax sequences (Section 4.3, Rule 4). What is *not* measured is whether an adapter's ΔW_q keeps that equivalence — the bit-identity result was obtained with the base weights only.

6.2 The Bridge Protocol: Targeted Re-Encoding

When a specialised adapter is needed only for a localized sub-routine — repairing code or symbolic arithmetic at steps $\hat{t} \ldots \hat{t} + k$ — before returning to general causal generation, retaining adapter overhead across the remaining sequence is wasteful. The **Bridge Protocol** treats adapter generation as an isolated, temporary patch:

1. **Surgical truncation.** The cache is cropped to the recorded absolute token boundary before step $\hat{t}$, preserving verified prefix representations. A clean base-prefix copy is retained separately from the specialist's cache.
2. **Specialist execution.** The task-specific adapter is engaged to generate the localized patch tokens $S_{\text{patch}} = \{x_{\hat{t}}, \ldots, x_{\hat{t}+k}\}$.
3. **Distribution re-alignment (the bridge).** The adapter is disengaged. Rather than flushing the context window or retaining adapted latent vectors, the raw token IDs of S_{patch} pass through a non-generative forward prefill under the frozen base weights, starting from the preserved base-prefix copy. The specialist's working cache is discarded. This rebuilds patch keys and values under the base policy.
4. **Resumption.** Autoregressive decoding resumes under the base policy at step $\hat{t} + k + 1$ from the bridge's output logits, without feeding the final patch token twice. Dependent artifacts downstream of a repaired node must be revalidated or regenerated. Independent artifacts may be preserved, but their later cached states must be replayed if their prior attention context included the changed prefix.

The economic argument is that prefill over a short window is parallel, so re-encoding k tokens costs one forward pass rather than k sequential decode steps, and the specialist's weights can be de-allocated immediately. That argument is plausible but **its cost is not established here**, and Section 4.4 cuts against the optimistic reading: at $H = 10$ a 90% reduction in forwarded tokens bought only $1.17\times$ wall-clock,

because launch and prefill overhead dominate at this scale. Any claim that the bridge is "imperceptible" or "single-digit milliseconds" would therefore contradict a measurement already in this paper. It has to be timed.

What would count as evidence. The protocol is worth adopting only if, against query-restricted persistence (Path 1) on the same rows, it satisfies all three:

1. **Cost:** re-encoding + adapter swap is cheaper in wall-clock than keeping the adapter resident for the remaining $T - (\hat{t} + k)$ steps. Tokens forwarded is the wrong metric; Section 4.4 shows it decouples from latency.
2. **Quality:** end-to-end recovery is not lower. The bridge rewrites the patch into the base distribution, so the failure mode to watch is a specialist edit that the base model then treats as out-of-distribution and talks itself out of.
3. **Coherence:** $\max |\Delta\text{logit}|$ between bridged and fully-base decoding over the same token sequence satisfies a predeclared, dtype-specific tolerance, with finite logits and matching continuation. The original 4.86×10^{-5} is an observed base-only maximum, not a universal adapter tolerance. The CPU fp32 integration below uses 10^{-4}.

If (1) fails, the protocol collapses to Path 1 with extra machinery. If (2) fails, the adapter was load-bearing for more than the patch and targeted re-encoding is the wrong abstraction. Either outcome is a result; neither is currently known.

6.3 Preliminary Coder Integration and Validation Readiness

A CPU fp32 prototype connects the existing commitment controller to one coder LoRA (`checkpoint-1000`, parent revision `c1899de289a04d12100db370d81485cdf75e47ca`). Task skills and deterministic verifiers are supplied explicitly; no learned code localizer or router is used. In a five-node base/coder/base/coder/base sequence, four policy transitions were verified across 112 LoRA layers and four nodes were committed. The coder failed the clamp task (2/5 function cases), which the controller rejected; the even-sum task passed (4/4 cases) and was bridged back to the base. The base diagnostic passed both coding tasks. These observations establish neither a specialist quality gain nor the cause of the clamp regression.

All accepted-node Bridges in that sequence had exact retained-prefix K/V, finite logits, matching next-token argmax, correct cache length, and matching four-token greedy continuations against fresh base prefill. The largest absolute logit delta was 5.20×10^{-5} against the predeclared fp32 tolerance 10^{-4}. An explicitly injected bad addition function was also rejected, repaired by the actual coder, and bridged successfully. That control tests the mechanism; it is not natural base-failure recovery. Source records: `results/cgr_adapter_switching/20261029T213116Z_df30837b.json` and `results/cgr_adapter_switching/20261029T213212Z_4e3f62d0.json`.

A separate dependency/validator package exercises delayed failure, descendant invalidation, preserved-artifact replay planning, parameterized SQL behavior, and static HTML contracts on 20 public parameter variations of one sanitized-card task family. Its passing fixture checks use zero model generations and supply no multi-domain efficacy evidence. Real frontend behavior, multi-adapter cache tests, independent task evaluation and the full-prefix-versus-switched-specialist ablation remain pending. The prospective protocol is `ADAPTER_VALIDATION_PROTOCOL.md`.

6.4 Two-Specialist Dependency-Graph Development Test

A live four-node derived card fixture uses one pinned chat-capable `Qwen/Qwen3-0.6B` backbone with both cyber and coder adapters. Its generated cyber SQL passed the parameterized-query contract; the result's row count was then checked using a generated coder factorial function. Both artifacts were committed to `DependencyGraph` after verification and base-bridge comparison, and the final composed check passed. The

cyber and coder bridges differed from fresh-base reference logits by at most 4.863739×10^{-5} and 3.337860×10^{-5} respectively, below the fp32 tolerance of 10^{-4}; argmax and four-token continuations matched. The 31.947-second CPU time includes base diagnostics, specialist attempts, verifiers and reference checks; initial model loading took 7.855 seconds separately. This is one disclosed development example, not a latency comparison.

The base also passed both node contracts in that sequence, so it does not demonstrate that specialists were necessary.

A second four-node graph used the original `card_00` clamp and defensive SQL contracts. The base passed clamp; the coder initially failed but passed after one retry whose prompt contained the verifier's failed cases. The cyber query then passed in the accumulated conversation, both exact-output base bridges passed with maximum logit differences 5.245209×10^{-5} and 5.483627×10^{-5}, and the final check consumed both outputs: a SQLite row for Alice and the correctly clamped value -10. The wall time was 25.229 seconds excluding 10.209 seconds loading, including diagnostics, the failed attempt and verifier feedback. This is a disclosed coder-failure retry, not a natural base-failure repair, a blind evaluation or the original 20-node delayed-failure/replay milestone.

An independent factorial task produced a genuine base error and a coder output passing three predefined cases; its accepted-token base bridge differed from fresh prefill by at most 4.434586×10^{-5}. However, attempting the cyber task after this repair failed: the cyber adapter repeated the code instead of producing JSON. On the initial no-feedback `card_00` clamp attempt, the base passed and coder failed, so that attempt was rejected. A bounded eight-task search after the cyber bridge found three base failures and no verified coder repairs. The successful derived graph is not the original 20-node delayed-failure fixture or evidence of a complete multi-domain recovery; the arithmetic experiments elsewhere still use the separate `Qwen3-0.6B-Base` backbone. Full records and timing definitions are in `findings/2026-10-30-coder-cyber-bridge.md`.

6.5 Cross-Dataset Cyber Routing and Live Full-Prefill Verification

We trained binary correctness heads on a 200-item single-answer SecEval sample, where the base answered 124 correctly and 76 incorrectly. The inputs were the final four generated answer-token hidden states, with standardization and an eight-dimensional SVD fitted solely on the training set. A `LinearStep` and a `TransformerStep` were adapted to binary classification; the mean of their sigmoid scores below 0.50 selected the cyber adapter. The heads were trained, but the base and the step-2500 cyber LoRA were frozen during inference. The score is not established as a calibrated correctness probability.

CyberMetric-80 arm	Correct	Accuracy
Base (Qwen3-0.6B)	54/80	67.50%
Cyber adapter	54/80	67.50%
SecEval-trained router	**57/80**	**71.25%**
Oracle, choose a correct arm if available	61/80	76.25%

The router selected the adapter on nine questions, producing three useful switches and zero harmful switches. These are sample-level observations, not a proof of safety. On the disjoint 40-question SecEval sample, base/cyber/router/oracle scored 26/23/26/29; one switch helped and one harmed. In the reverse direction, heads trained on all 80 CyberMetric items evaluated on 100 SecQA items gave base/cyber/router/oracle scores of 97/90/97/97, with nine switches and neither useful nor harmful switches. Strict answer-format parsing counted 2 CyberMetric, 5 SecEval and 7 SecQA cyber responses as invalid; those must not silently be treated as correct.

The live CyberMetric loop regenerated the base output, computed the frozen router score, and, when selected, discarded the previous generation cache and generated from the complete prompt with the cyber adapter. It reproduced all 80 saved base outputs and routing decisions and all nine selected standalone cyber outputs. A forward hook verified that each specialist's initial forward received the full prompt and a cache of length zero. This demonstrates the specified full-prefill behavior and outcome on these questions. It does not verify a multi-step DAG, classifier calibration, safe commit decision, or specialist-to-base bridge. The SecEval and CyberMetric material is public and CyberMetric had already been examined; exact and high character-similarity checks do not eliminate paraphrases or unknown pretraining/adapter contamination. The +3/80 gain is exploratory and does not establish generality or safety. Artifacts: `experiments/cyber_router_crossdataset.py`, `experiments/cyber_router_live.py`, `results/cyber_router_crossdataset_v1.json`, and `results/cyber_router_live_cybermetric80_v3.json`.

6.6 Experience Memory and UI: Exploratory Results

An in-memory Chroma store with MiniLM embeddings retrieved a short formatting rule for a 20-item CyberMetric sample. The cyber arm moved from 13/20 with two format failures to 16/20 with none; the base moved from 15/20 to 16/20. The rule was written after observing those failures, and the other questions had been examined, so this does not establish held-out generalization or an advantage of retrieval over directly including the same instruction. Scoped, expiring memory was tested as a component, not integrated into the live routing loop.

Experience Memory as a Semantic Bridge: Controlled Development Test

We investigated whether a scoped experience rule could help the cyber adapter follow its output contract after seeing coder-generated text. We reconstructed three previously verified coding contexts (factorial, clamp with verifier feedback, and even-sum) and combined each with two related SQLite tasks (selecting a user by name or ID). The cyber adapter used fresh full-prompt prefill in every condition, with the same greedy settings. The memory rule, written after observing the factorial failure, instructed the model to output only a JSON object containing `sql` and `params`. Chroma retrieved the rule using MiniLM embeddings within an explicitly supplied task scope; the memory tag was inserted as text in the user message, not as a privileged system instruction.

Condition	SQL contract passed	Bare JSON and contract passed	Repeated Python
Cyber task alone	6/6	0/6	0/6
After coder output, no rule	2/6	2/6	4/6
After coder output + direct rule	6/6	6/6	0/6
After coder output + retrieved rule	6/6	6/6	0/6

The standalone outputs passed the SQL behavior contract after removal of an outer Markdown fence but did not satisfy strict bare-JSON formatting. The same two standalone prompts were repeated across the three context rows, so 6/6 is not six independent standalone tasks. Without the rule, the factorial and even-sum contexts led to repeated Python on both SQL tasks; the clamp-feedback context did not. Adding the rule was associated with passing outputs on these development cases, but the study did not isolate whether code content, chat structure, context length or another factor caused the initial failures.

The direct and retrieved conditions had identical prompt token IDs and identical outputs in all six comparisons. **This demonstrates scoped retrieval and application of a manually authored rule in these cases; it does not establish that the memory system automatically detected the error, discovered the rule, or outperformed directly supplying it.** The rule was created after the observed failure, scope was supplied by the caller, and the six combinations are not an independent held-out benchmark. A future semantic-bridge study should evaluate multiple candidate rules, irrelevant and expired

memories, and unseen task families. Complete inputs, outputs and checks are in `results/cyber_context_memory_controls_v1.json`; the protocol and limitations are in `findings/2026-10-30-cyber-context-memory-controls.md`.

Five-rule ablation. We next stored five handcrafted SQL, Python, math, UI and phishing rules in the same supplied scope and queried Chroma with the current task text. The SQL rule ranked first for all three SQLite prompts; four hand-authored non-SQL control queries also selected their intended rule. After the saved coder factorial output, the no-memory cyber arm passed 0/3 SQL checks and the retrieved-rule arm passed 2/3. The remaining output was valid JSON but selected a non-existent `user_id` column; its reworded prompt described "user id" without explicitly specifying the `id` schema, so the failure is ambiguous. These are three related development prompts, not independent held-out tasks.

A poem query with no active scope retrieved no rule. This isolation depended on the caller-supplied scope: when the same poem was queried within the five-rule scope, top-1 retrieval returned the irrelevant math rule under the then-ungated implementation. Thus the negative control does not establish automatic abstention or context-contamination detection. The prior direct-instruction comparison remains essential: matching retrieved and direct prompt tokens produced matching outputs. See `findings/2026-10-30-multirule-memory-ablation.md` and `results/multirule_memory_ablation_v1.json`.

Strict similarity gate. A subsequent implementation adds the requested rule that a retrieved candidate must have cosine similarity strictly greater than 0.90. With this cutoff the poem query was rejected even when the five-rule scope was supplied, and an exact match to the stored SQL description was retained. However, all three ordinary SQL prompts were also rejected: their relevant-rule similarities were approximately 0.575, 0.568 and 0.642. The four other domain-control queries also fell below 0.90. This illustrates the false-abstention cost of an uncalibrated cutoff; cosine similarity is not a probability of correctness and can range from -1 to 1. The gate-only check did not generate new model outputs, so the earlier ungated generation table remains a separate experiment. The threshold defaults to 0.90 and can be configured; the earlier scripts explicitly use an ungated setting to reproduce their original protocol. See `results/multirule_memory_gate_090_v1.json`.

A paired FocusFlow landing-page prompt produced complete HTML from both the base and the final UI adapter, with identical prompt tokens and decoding settings. Both omitted the requested functional signup, working hero action, schedule preview and color palette; the adapter output also lacked an associated email label and its mobile layout expanded beyond the target width. One example is not a UI benchmark. The UI data's React/Next.js component emphasis is a plausible mismatch with the self-contained HTML task, not a demonstrated cause.

---

7. Conclusion

The measured arithmetic repair loop operates without fine-tuning the base language model at inference or relying on text-based self-correction prompts. By reading un-pooled intermediate hidden states, enforcing physical logit exclusion, and surgically cropping the key-value cache, language models can detect and repair reasoning divergence at a fraction of the computational cost of regeneration: 1.59x the recovery rate of a full rewrite on a blind 331-chain vault, with the repair's generation step 26x faster than regenerating the chain.

On the reported arithmetic workload, cache-truncated regeneration reduces the measured generation slice from about 4.5 seconds to below 200 milliseconds; this timing includes generation after the crop and is not the cost of the crop alone. The full loop, including observer inference, runs at $1.7\text{-}2.0\times$ the speed of a rewrite in the reported arithmetic experiments. A future multi-step evaluation must test whether dependency-aware replay, verification and specialist routing preserve that advantage on real tasks and different model and hardware configurations.

What this paper does not establish. Four boundaries are load-bearing, and each is stated where the relevant experiment appears:

1. **No live 20-node delayed-failure graph.** The dependency-graph results in §6.3–§6.4 are four-node development fixtures with deterministic verifiers; the original 20-node detect/invalidate/replay milestone has not been run.
2. **No claim that specialists outperform the base.** In the two-specialist sequence the base passed both node contracts; the coder failed the clamp task (2/5 cases) while the base passed (5/5); and an eight-prompt search after a cyber bridge found three base failures and zero verified coder repairs (§6.4).
3. **No unified multi-domain DAG.** Every arithmetic result uses `Qwen3-0.6B-Base`; every adapter, bridge and routing result uses the chat-capable `Qwen/Qwen3-0.6B` at revision `c1899de289a04d12100db370d81485cdf75e47ca`. No experiment spans both backbones, so the system is an integration of separate experiments, not one measured pipeline.
4. **No measured multi-checkpoint memory bound.** The recorded peaks — 2,293.5 MB and 2,328.0 MB — are single-0.6B-checkpoint runs. The ~3.4 GB two-checkpoint figure in §6 is arithmetic from checkpoint sizes before activations, not a measured peak.

The routing gain is +3/80 on a benchmark that had been examined before the experiment, contamination is unknown, and fp16 Bridge tolerance is untested.

References and research artifacts

- Tihanyi et al., CyberMetric: A Benchmark Dataset Based on Retrieval-Augmented Generation for Evaluating LLMs in Cybersecurity Knowledge, IEEE CSR 2024; dataset.
- XuanwuAI, SecEval cybersecurity benchmark and dataset.
- Liu et al., SecQA: A Concise Question-Answering Dataset for Evaluating Large Language Models in Computer Security; dataset.
- Qwen team, Qwen3-0.6B chat model and Qwen3-0.6B-Base.
- Project source records and model/data revisions are indexed in `REPRODUCIBILITY.md` and in the linked findings and result files.

Errata

2026-10-29 — the vault Step-4 column was a precision, not a recall. `experiments/eval_vault.py` called `per_step_recall(v_lab, pred)` and `balanced_accuracy(v_lab, pred)` where both signatures are `(pred, lab)`. Every `*_rec` array written to `results/verify_vault_h5.json` is therefore per-step *precision*, and every `*_bal` value is a macro-precision. The consequence is the sentence this paper used to carry — that the Two-Tier rule fell to `0.410` Step-4 recall and underperformed both single probes — which is the opposite of what the arm does on recall.

The call order is fixed; `per_step_precision` is now a separate function so the two metrics cannot be conflated again; and `eval_vault.py --diagnostics-only` recomputes the localizer block without touching the stochastic arm results, refusing to splice a drifted fit over the original. The corrected block is `results/verify_vault_h5_localizer_refit.json`; `results/verify_vault_h5.json` is annotated in place and **none of its numbers were changed**.

A re-fit was necessary rather than a re-label because the 2026-10-28 run never stored its predictions. The re-fit is deterministic on this machine (two runs bit-identical) but is not the same fit: it reproduces Linear and Transformer Top-1 to within `0.01`, while the two threshold-dependent arms move by `0.027`. That drift

concentrates exactly at the `P_trf(4) \ge 0.35` boundary, which is itself the evidence for the reframing in §4.3 — a hard cutoff on one class posterior is numerically brittle as well as mistuned. The recovery arms (`46/331`, `23/331`, `73/331`), the paired tests, and the systems block were **not** re-run and are unchanged.

2026-10-29 — §1.3 described a control that does not exist. The section reported `+0.105` "vanishing when matched against parameter-equivalent LoRA adapters (`8.15M` vs `442K`)". Three problems: `8.15M` vs `442K` is not parameter-equivalent — it is the *unmatched* comparison, and it produced `+0.135`, not `+0.105`; the matched control is `22.3M` vs `22.3M` and produced `+0.035 [-0.045, +0.115]`, not significant; and `+0.105` does not appear in any artifact in `results/`. §1.3 now reports the matched control from `findings/2026-10-25-steering-capacity-control.md`, and the unmatched `+0.135` is named as what it was.

Both corrections change conclusions rather than wording, which is why they are listed here instead of being edited away.

Reproducibility & Source Verification

All code, trained weights, raw outputs, and evaluation logs are deterministic and tracked:

- **Harness & Pipelines:** `experiments/path_a_loop.py`, `experiments/eval_vault.py`, `experiments/interp_calls.py`
- **Raw JSON Metrics:** `results/path_a_loop_h5.json`, `results/verify_vault_h5.json`, `results/verify_vault_h5_localizer_refit.json`, `results/verify_h10_scaling.json`, `results/oracle_rewind_h5.json`
- **Underlying Corpus:** `results/arith_steps_h5.pt` (tuning), `results/arith_steps_h5_vault.pt` (blind vault)
- **Detailed Finding Reports:** `findings/2026-10-28-out-of-sample-and-h10-verification.md` (vault + H=10, authoritative for the systems and generalization numbers above); `findings/2026-10-28-path-a-ensemble-kv-slicing.md` (Path A; corrected in place on 2026-10-28 after the vault audit); `findings/2026-10-25-steering-capacity-control.md` (capacity control cited in §1.3)